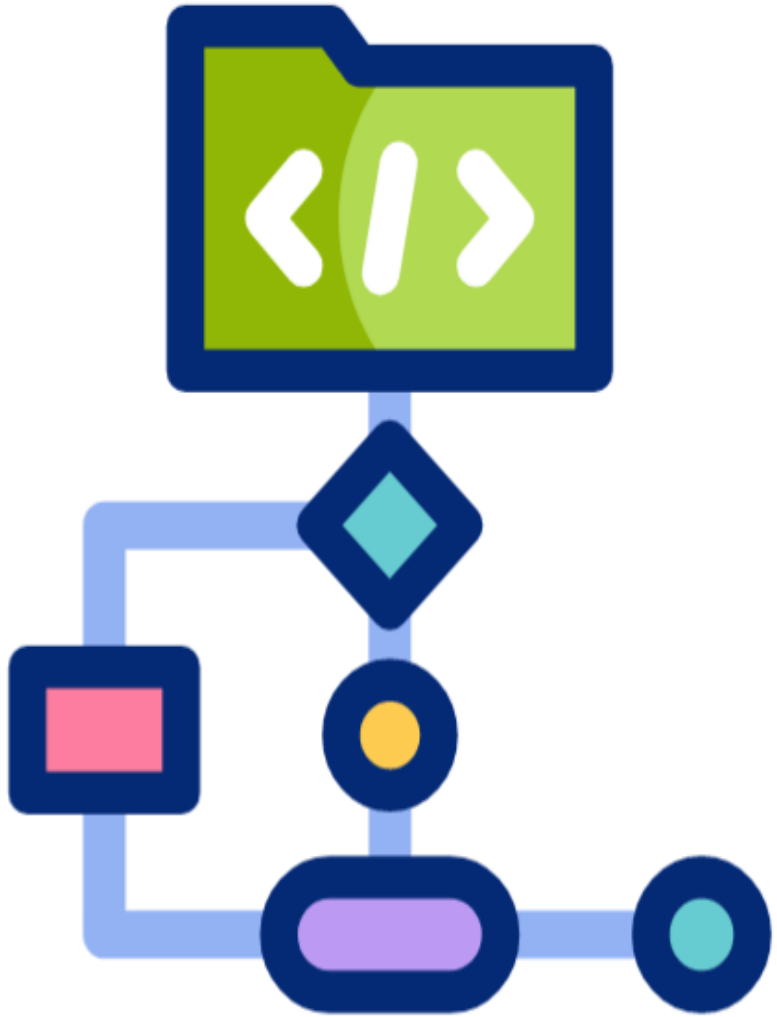
Algorithms

Algorithm

- It is a list of instructions specifying a precise description of a step by step process that terminates after a finite number of steps for solving an algorithm problem producing the correct answer in the end.
- It is a recipe for solving problems.
- A finite set of an instruction that specifies a sequence of operation to be carried out in order to solve a specific problem.
- An unambiguous procedure specifying a finite number of steps to be taken.



Properties of Algorithm

Finiteness - there is an exact number of steps to be taken and has an end.

Absence of Ambiguity - means that every instruction is precisely described and clearly specified.

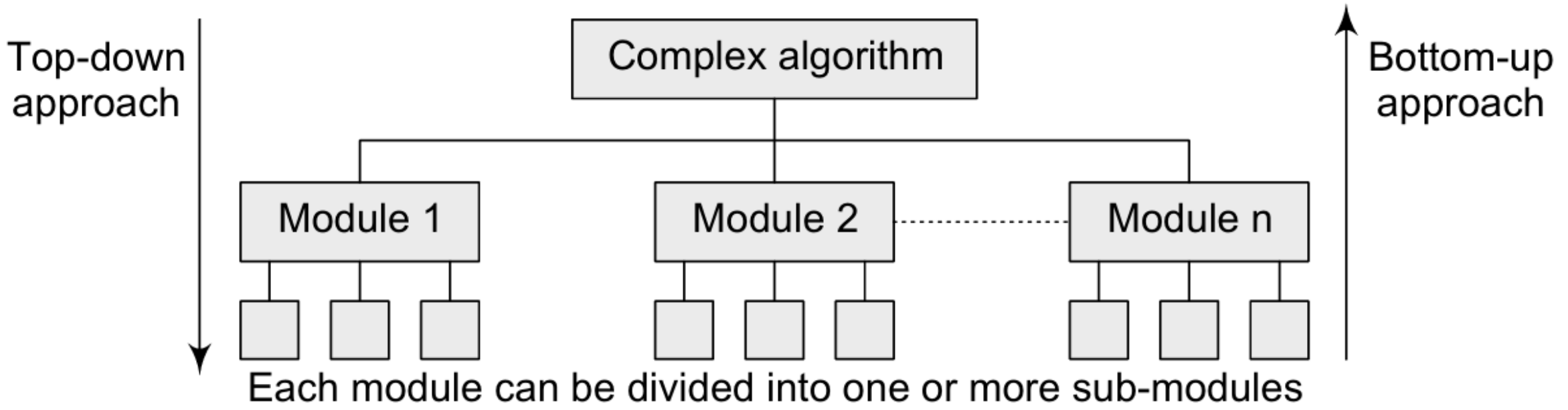
Sequence of Execution - instructions are performed from top to bottom.

Input and Output - defined the unknowns of the problem is specified and with the expected outcome.

Effectiveness - the solution prescribed is guaranteed to give a correct answer and that the specified process is faithfully carried out.

Scope Definition - applies to a specific problem or class of problem.

DIFFERENT APPROACHES TO DESIGNING AN ALGORITHM



A complex algorithm is often divided into smaller units called *modules*. This process of dividing an algorithm into modules is called *modularization*.

DIFFERENT APPROACHES TO DESIGNING AN ALGORITHM

Top-down approach

- ✓ A top-down design approach starts by dividing the complex algorithm into one or more modules.
- ✓ Top-down design method is a form of stepwise refinement where we begin with the topmost module and incrementally add modules that it calls.

Bottom-up approach

- ✓ In the bottom-up design, we start with designing the most basic or concrete modules and then proceed towards designing higher level modules.
- ✓ In this approach sub-modules are grouped together to form a higher level module.

CONTROL STRUCTURES USED IN ALGORITHMS

Sequence, it mean that each step of an algorithm is executed in a specified order.

Example: Algorithm to add two numbers

```
Step 1: Input first number as A
Step 2: Input second number as B
Step 3: SET SUM = A+B
Step 4: PRINT SUM
Step 5: END
```

Decision, Decision statements are used when the execution of a process depends on the outcome of some condition.
For example, Algorithm to test for equality of two numbers

```
Step 1: Input first number as A
Step 2: Input second number as B
Step 3: IF A = B
                PRINT "EQUAL"
        ELSE
                PRINT "NOT EQUAL"
        [END OF IF]
Step 4: END
```

CONTROL STRUCTURES USED IN ALGORITHMS

Repetition, which involves executing one or more steps for a number of times, can be implemented using constructs such as while, do–while, and for loops.

Example: Algorithm to print the first 10 natural numbers

```
Step 1: [INITIALIZE] SET I = 1, N = 10
Step 2: Repeat Steps 3 and 4 while I<=N
Step 3: PRINT I
Step 4: SET I = I+1
        [END OF LOOP]
Step 5: END
```

EXERCISE 1

Write an algorithm to find whether a number is even or odd.

```
Step 1: Input number as A
Step 2: IF A%2 =0
                PRINT "EVEN"
        ELSE
                PRINT "ODD"
        [END OF IF]
Step 3: END
```

EXERCISE 2

Write an algorithm to find the larger of two numbers.

```
Step 1: Input first number as A
Step 2: Input second number as B
Step 3: IF A>B
           PRINT A
        ELSE
           IF A<B
              PRINT B
        ELSE
              PRINT "The numbers are equal"
              [END OF IF]
        [END OF IF]
Step 4: END
```

EXERCISE 3

Write an algorithm for swapping two values.

```
Step 1: Input first number as A
Step 2: Input second number as B
Step 3: SET TEMP = A
Step 4: SET A = B
Step 5: SET B = TEMP
Step 6: PRINT A, B
Step 7: END
```

Methods of Specifying Algorithm

Pseudocode - specifies the steps of algorithm using essentially natural language of superimposed control structure.

Flowchart - a traditional graphical tool with standardized symbols. Show the sequence of steps in an algorithm.

Pseudocode

Write a pseudocode for swapping two values

```
BEGIN
   INPUT A, B
   SET TEMP = A
   SET A = B
   SET B = TEMP
   OUTPUT "After swapping: A = ", A, " B = ", B
END
```

Pseudocode

Write a pseudocode to find the largest number

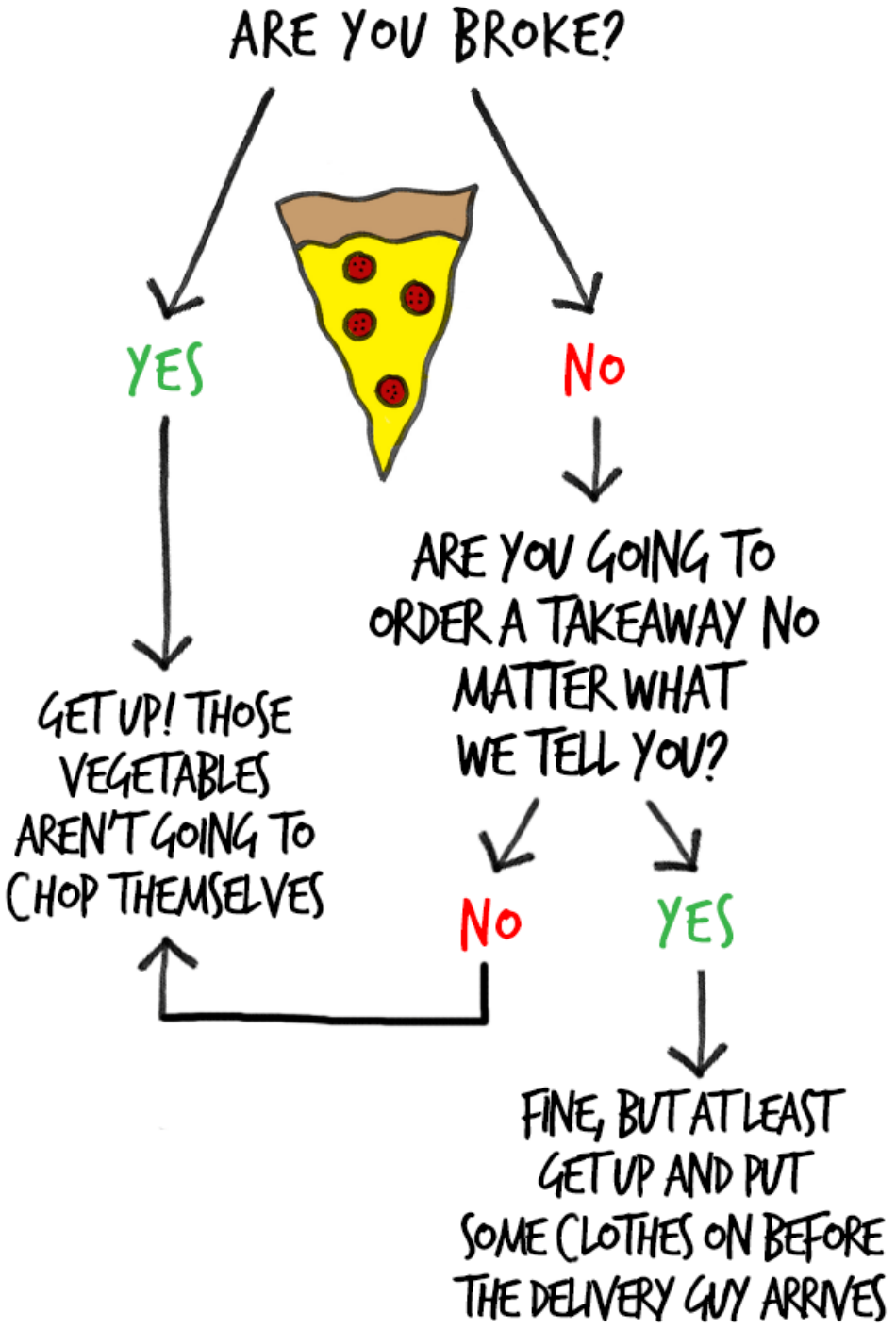
```
BEGIN
  INPUT numbers[1..n]        // an array of n numbers
  SET largest = numbers[1]   // assume first number is the largest

  FOR i = 2 TO n DO
    IF numbers[i] > largest THEN
      largest = numbers[i]
    ENDIF
  ENDFOR

  OUTPUT "The largest number is: ", largest
END
```

Flowcharting Guidelines

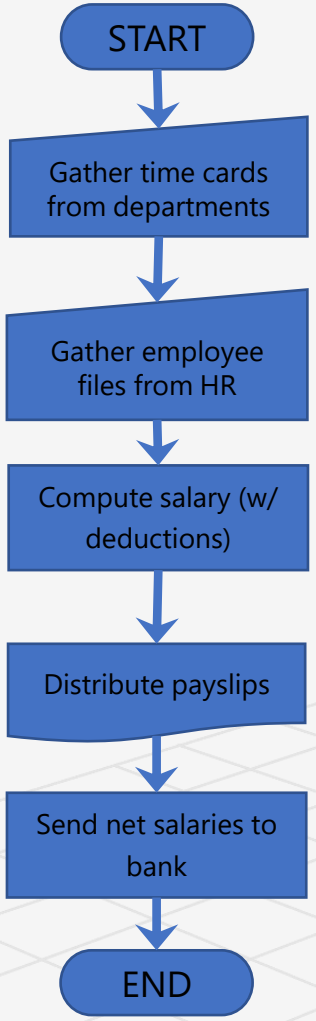
1. The flowchart should flow from top to bottom
2. If the chart becomes complex, utilize connecting blocks
3. Avoid intersecting flow lines
4. Use meaningful description in the symbol



Methods of Specifying Algorithm

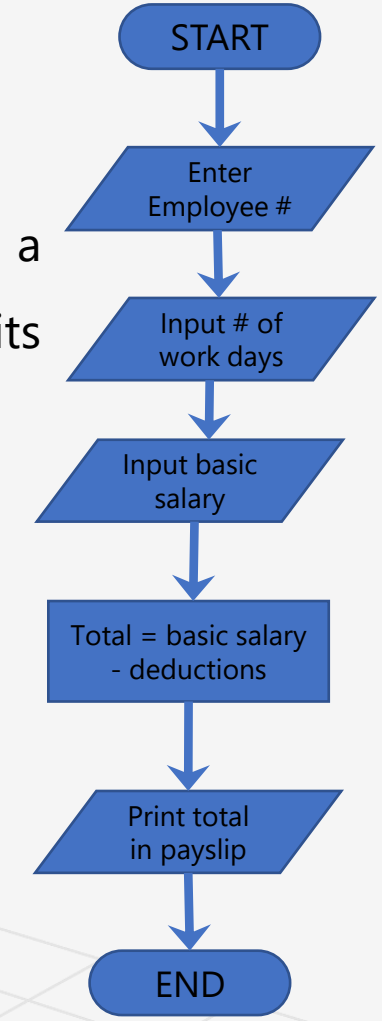
System Flowchart

A flowchart is a formalized graphic representation of a logic sequence, work or manufacturing process, organization chart, or similar formalized structure.

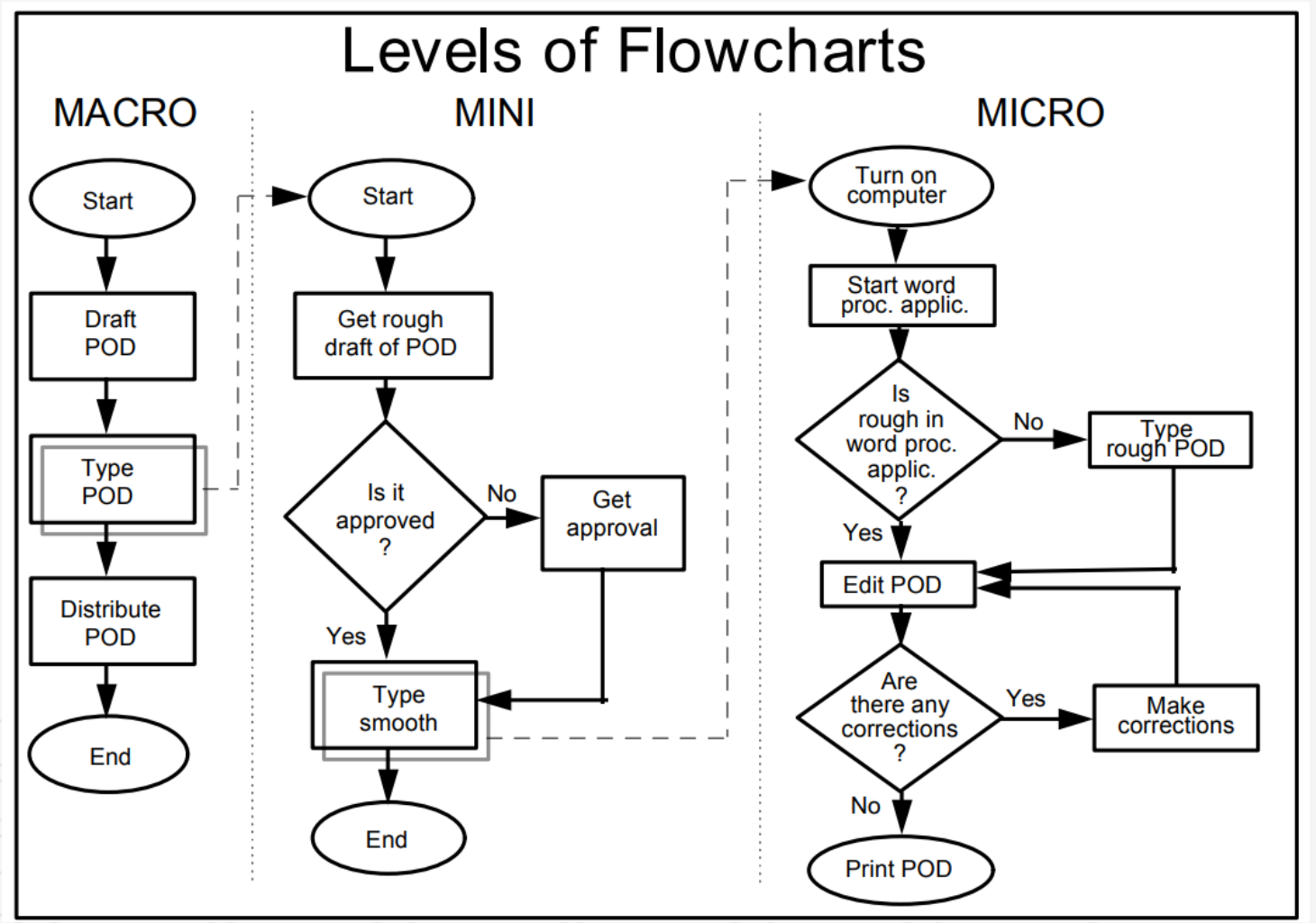


Program Flowchart

A graphical representation of a computer program in relation to its sequence of functions



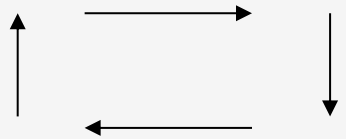
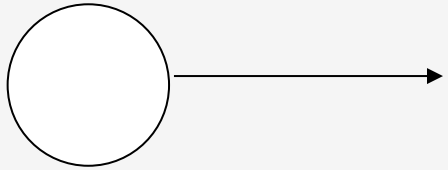
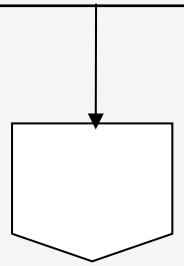
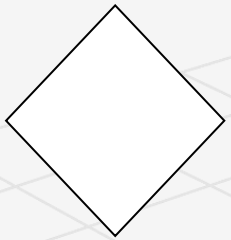
- **Macro level.** The top leadership may not need the amount of detail required by the workers in a process.
- The term "**mini" or "midi"** is used for a Flowchart that falls between the big picture of the macro level and the fine detail of the micro level.
- **Micro level.** People trying to improve the way a job is done need a detailed depiction of process steps.



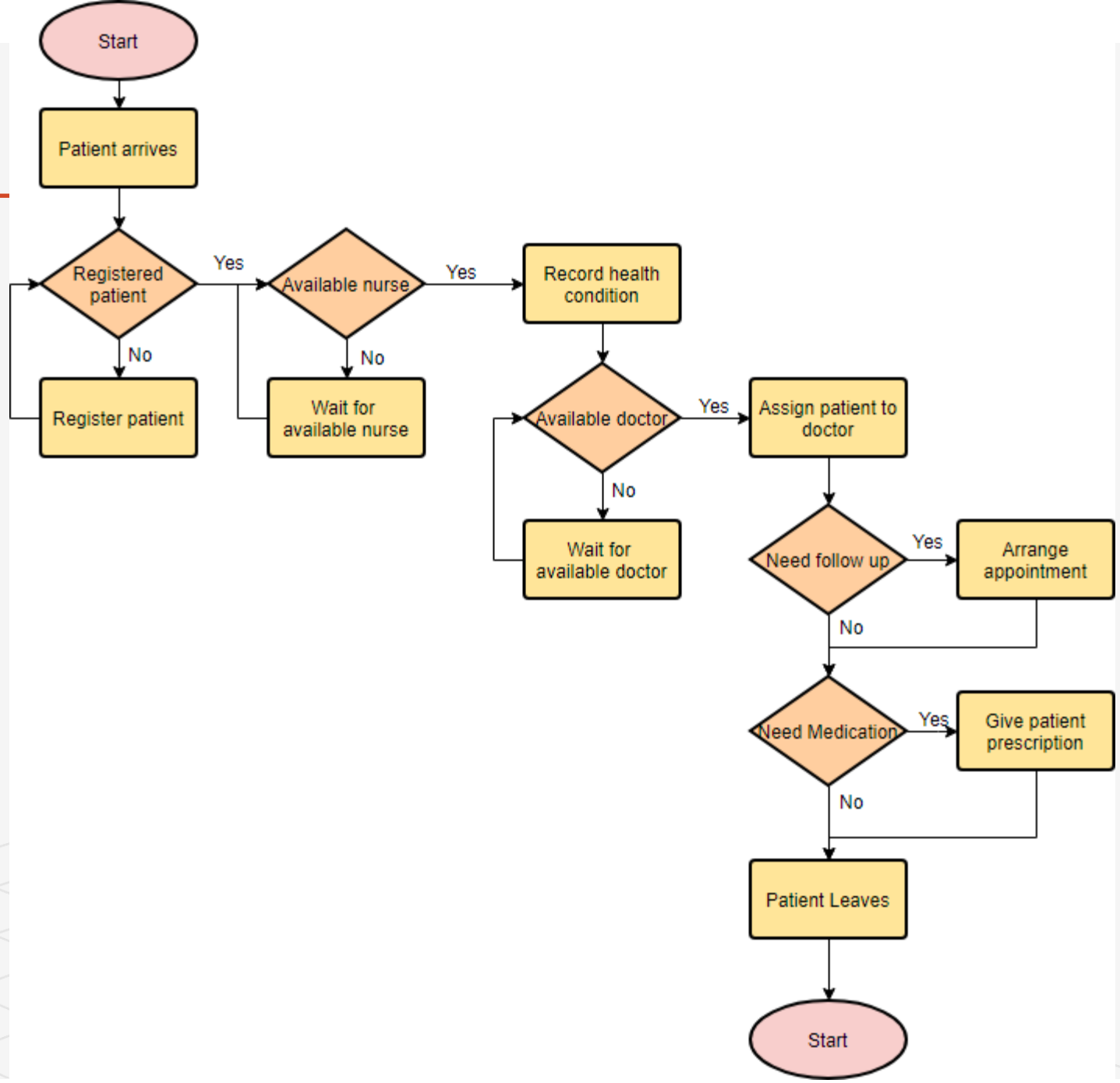
Basic Flowcharting Symbols

SYMBOL	NAME	DESCRIPTION
	**TERMINAL**	Defines the starting and ending point of a flowchart.
	**INITIALIZATION**	The preparation or initialization of memory space for data processing.
	**INPUT/OUTPUT**	The inputting of data for processing, and printing out of processed data.
	**PROCESS**	Manipulation of data(assignments and mathematical computations)

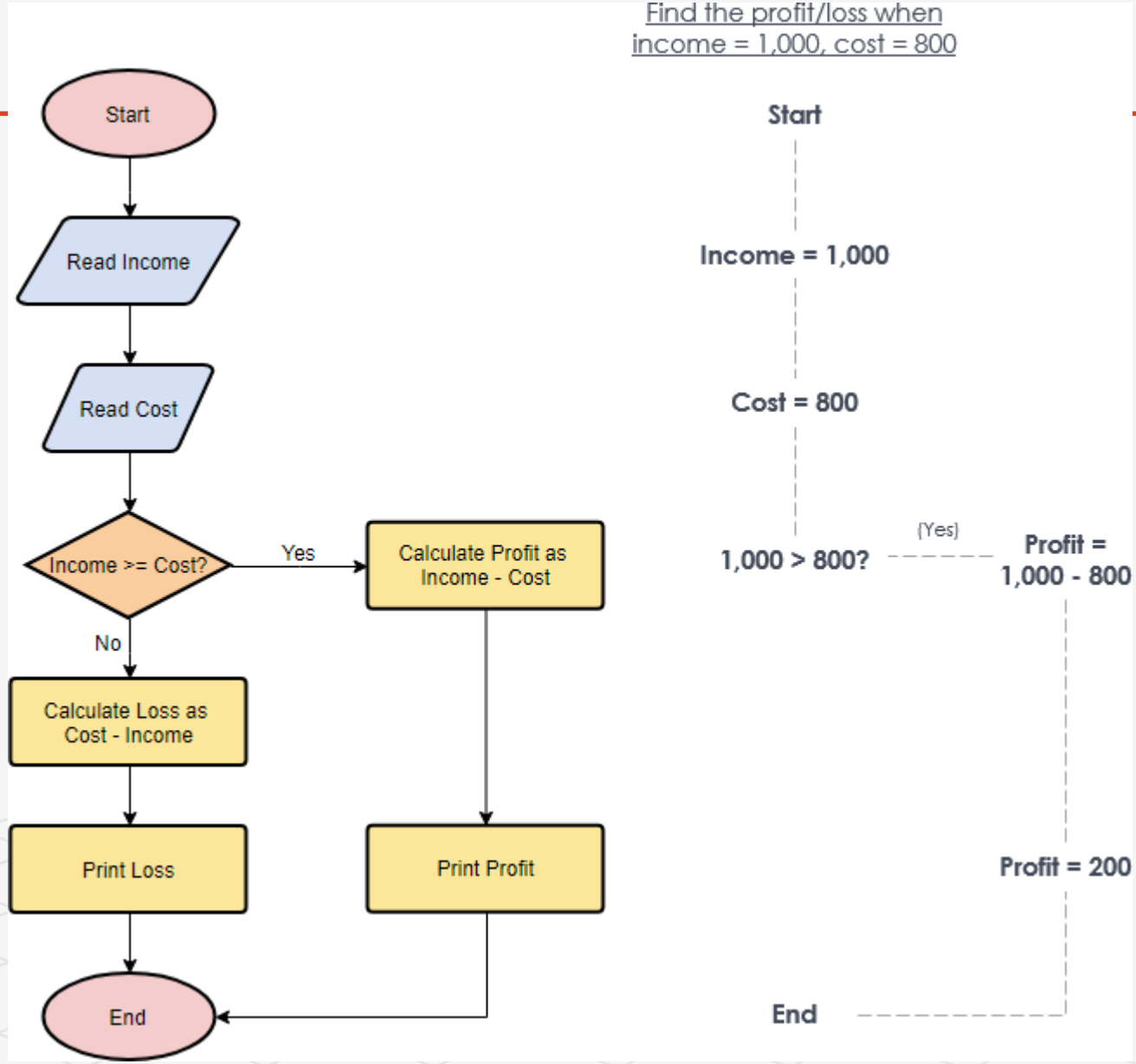
Basic Flowcharting Symbols

SYMBOL	NAME	DESCRIPTION
	**FLOW LINES**	Defines logical sequence of the program. Its points to the next symbol to be performed
	**ON-PAGE CONNECTOR**	Connects to the flowchart to avoid spaghetti connection on the same page
	**OFF-PAGE CONNECTOR**	Connects the flowchart on different page to avoid spaghetti connection
	**DECISION**	Process conditions using relational operators. Used for trapping and filtering data.

Medical Service



Calculate Profit and Loss



Branching And Control Of Flow



Flow Lines



On-page Symbol



Terminator Symbol



Off-page Symbol

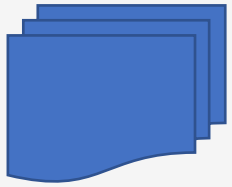


Decision Symbol

Input and Output



Data I/O Symbol



Multi-document Symbol



Document Symbol



Display Symbol



Manual Input Symbol

Process/Operation Symbols



Process Symbol



Pre-defined Process



Preparation Symbol



Delay

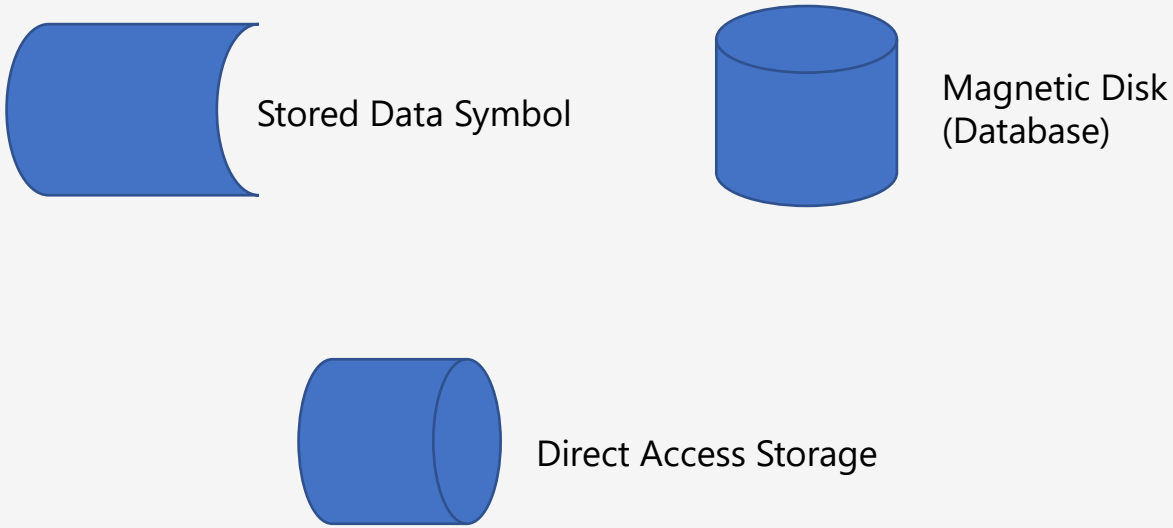


Alternate Process Symbol

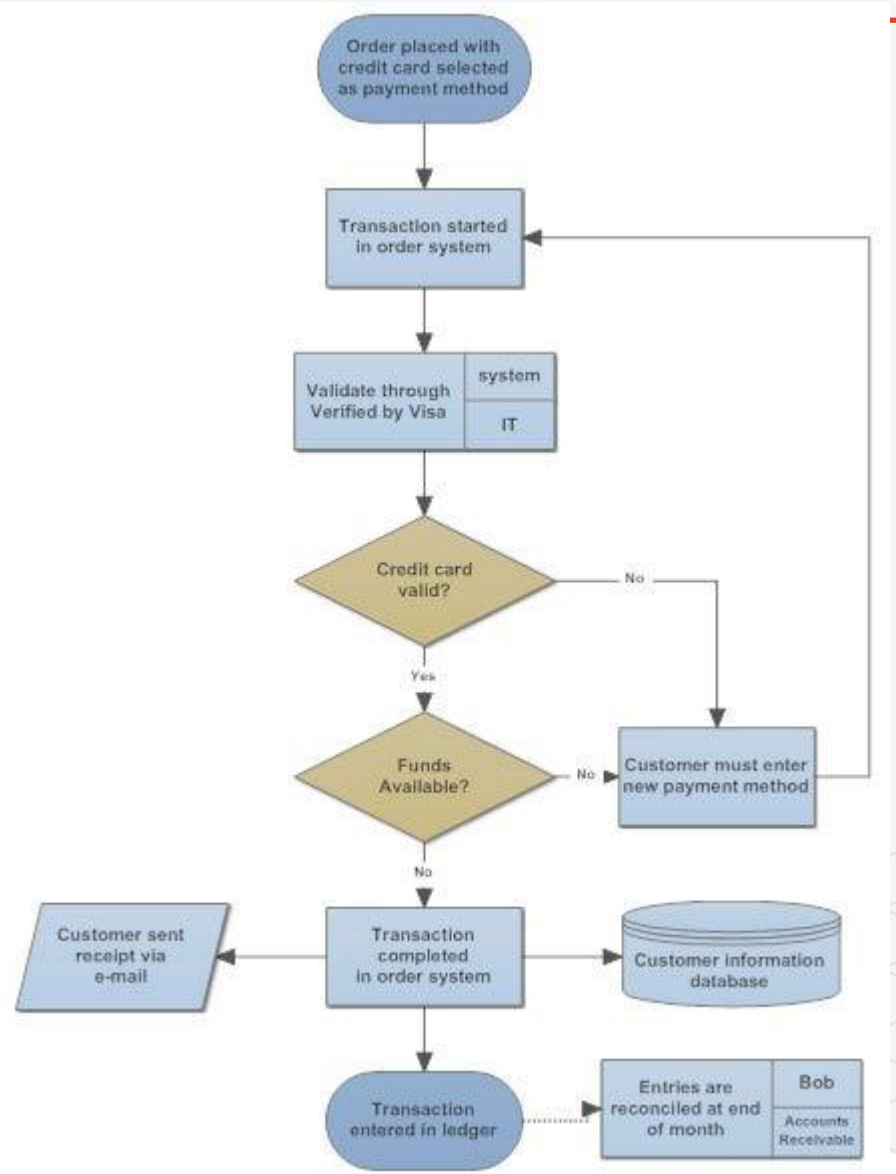


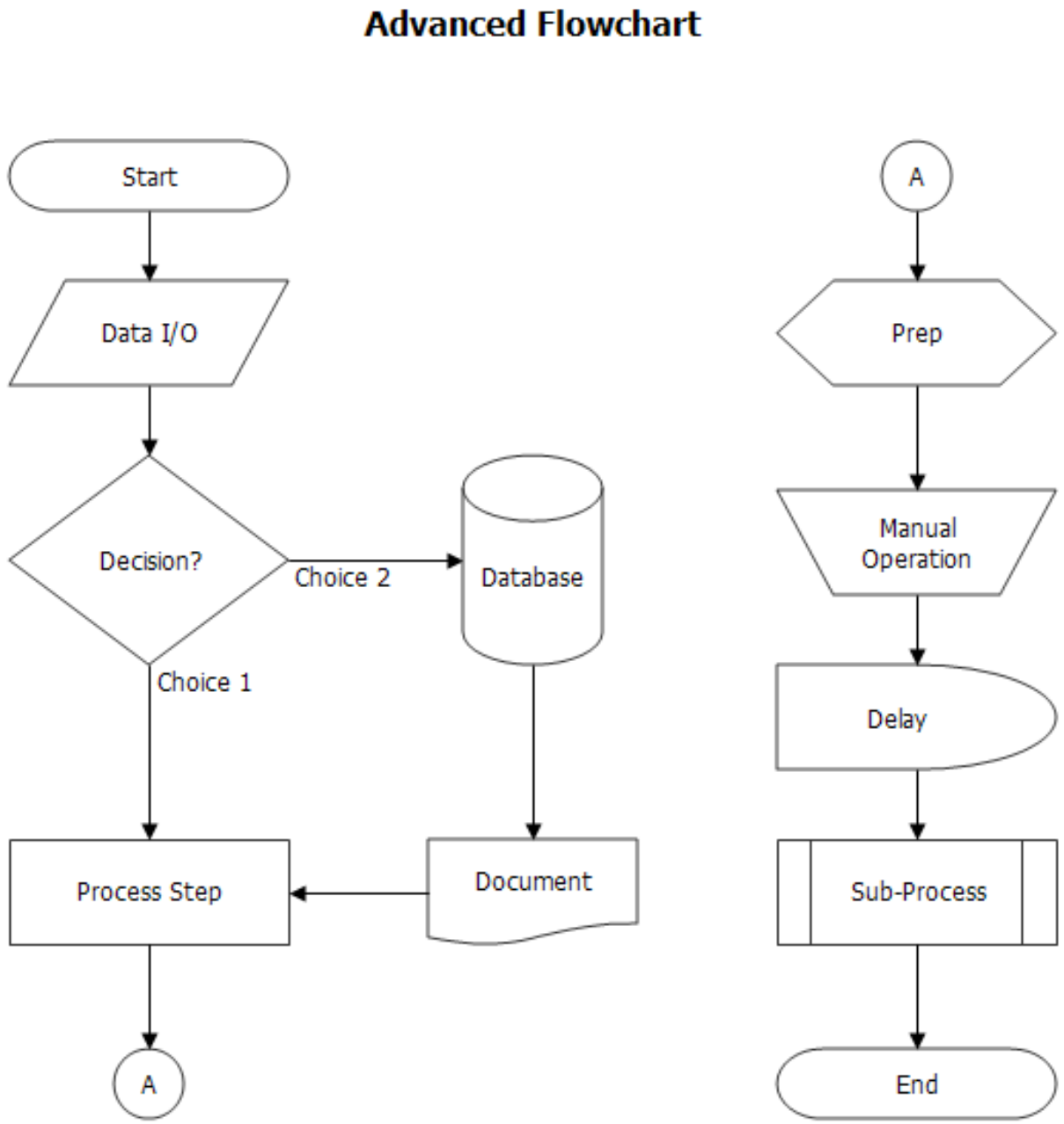
Manual Operation

File And Information Symbols



Credit Card Processing





Algorithm & Flowchart to convert temperature from Fahrenheit to Celsius

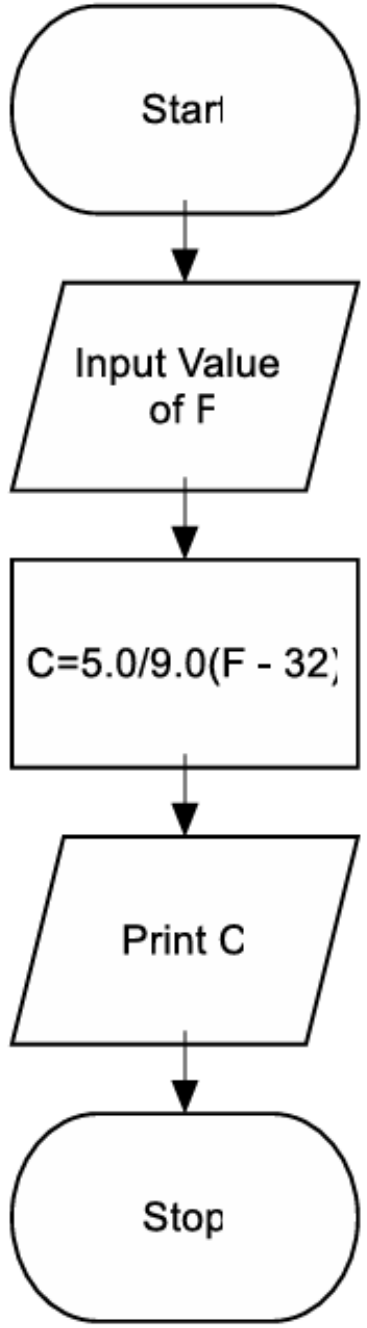
Step-1 Start

Step-2 Input temperature in Fahrenheit say F

Step-3 C = 5.0/9.0 (F - 32)

Step-4 Display Temperature in Celsius C

Step-5 Stop



Algorithm & Flowchart to find Area and Perimeter of Rectangle

Step-1 Start

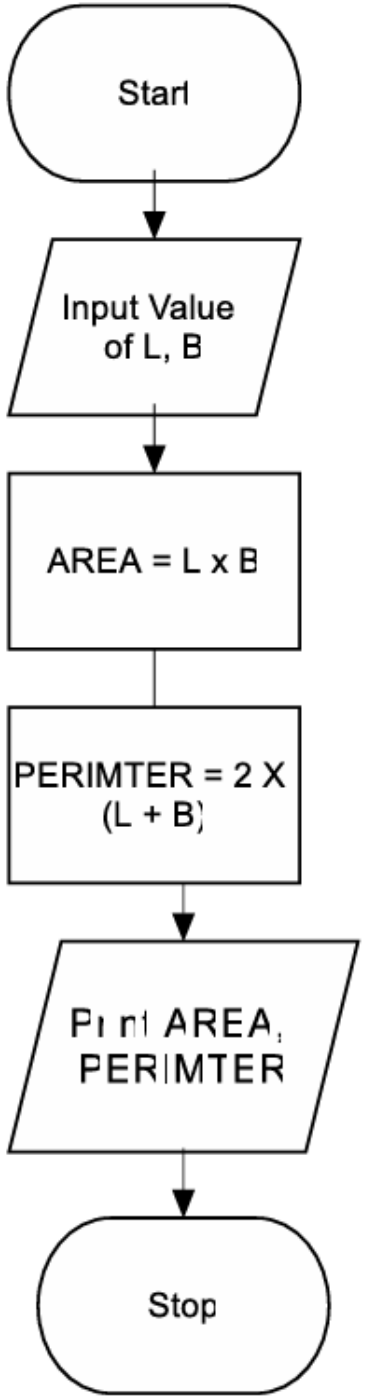
Step-2 Input Side Length & Breadth say L, B

Step-3 Area = L x B

Step-4 PERIMETER = 2 x (L + B)

Step-5 Display AREA, PERIMETER

Step-6 Stop



Algorithm & Flowchart to find the smallest of two numbers

Step-1 Start

Step-2 Input two numbers say

NUM1,NUM2

Step-3 IF NUM1 < NUM2 THEN

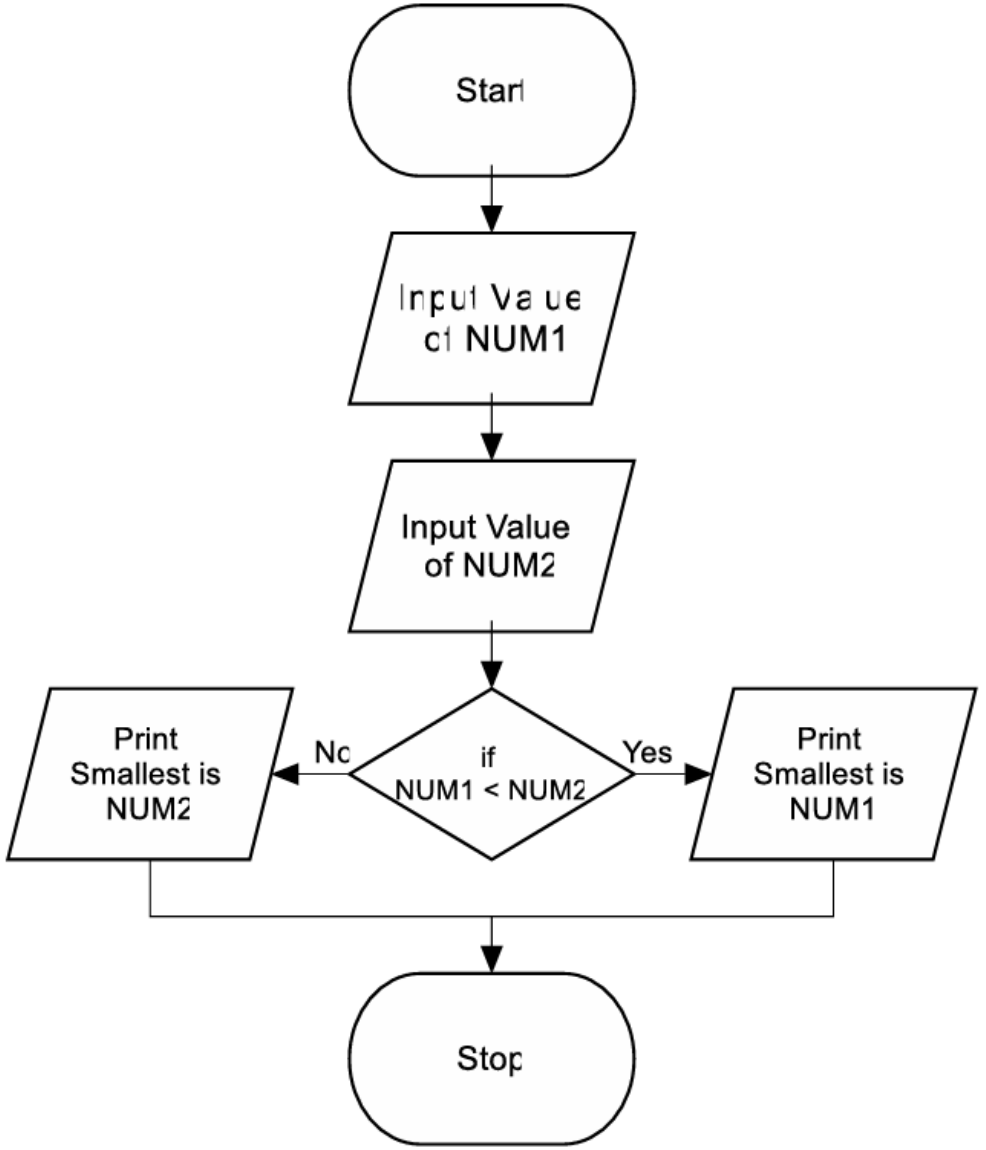
print smallest is NUM1

ELSE

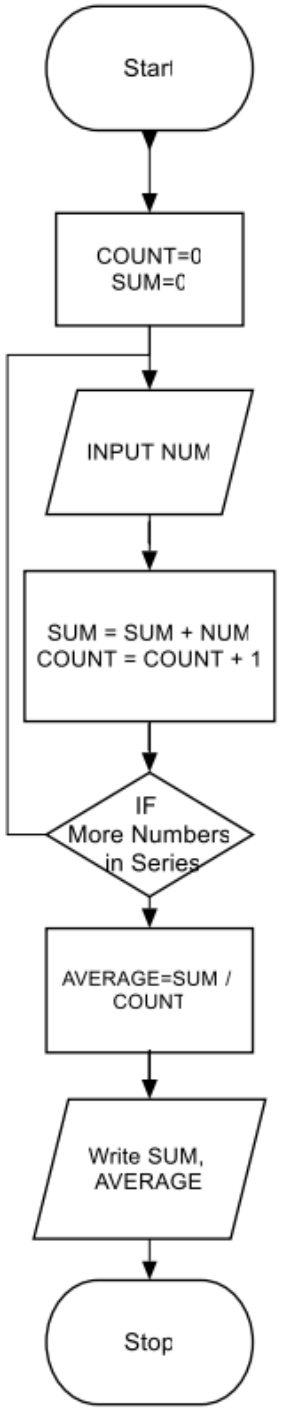
print smallest is NUM2

ENDIF

Step-4 Stop



Activity: Algorithm and Flowchart



Given the Algorithm. Draw a Flowchart to find sum and average of given series of numbers

Step-1 Start

Step-2 COUNT=0

Step-3 SUM=0

Step-4 Input NUM (next number in series)

Step-5 SUM= SUM +NUM

Step-6 COUNT=COUNT+1

Step-7 IF More Number in Series then

GOTO Step-4

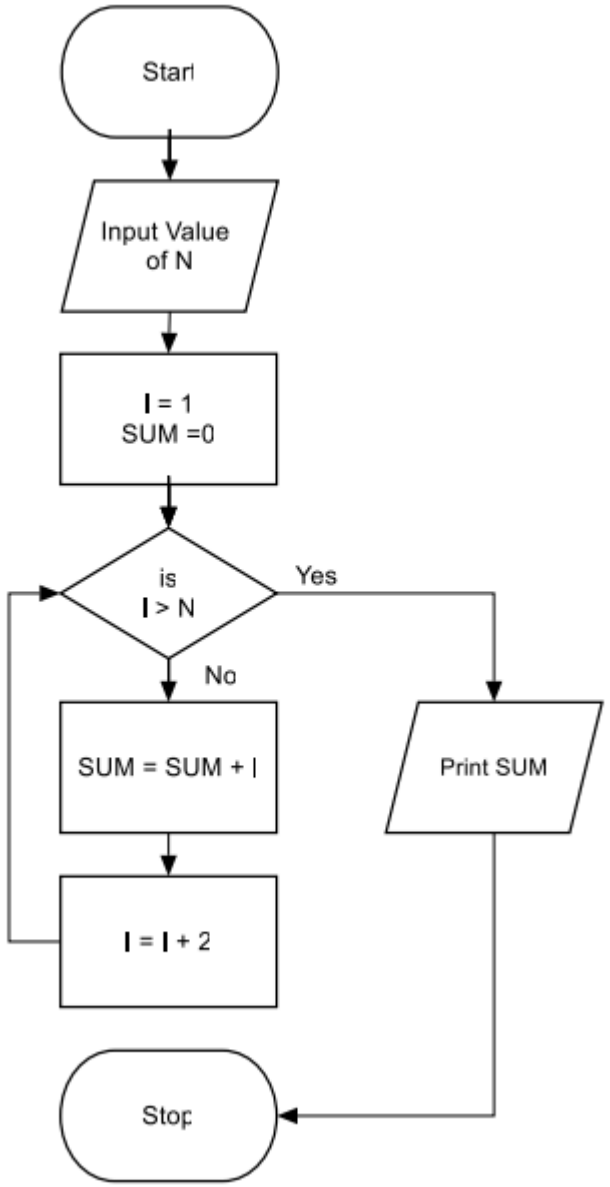
ENDIF

Step-8 AVERGAE=SUM / COUNT

Step-9 WRITE SUM, AVERAGE

Step-10 Stop

Given the Algorithm. Draw the flowchart to find sum of series 1+3+5+.....+N, Where N is positive odd Integer



Step-1 Start

Step-2 Input Value of N

Step-3 I = 1, SUM=0

Step-4 IF (I >N) THEN
GO TO step 8
ENDIF

Step-5 SUM = SUM + I

Step-6 I = I + 2

Step-7 Go to step-4

Step-8 Display value of SUM

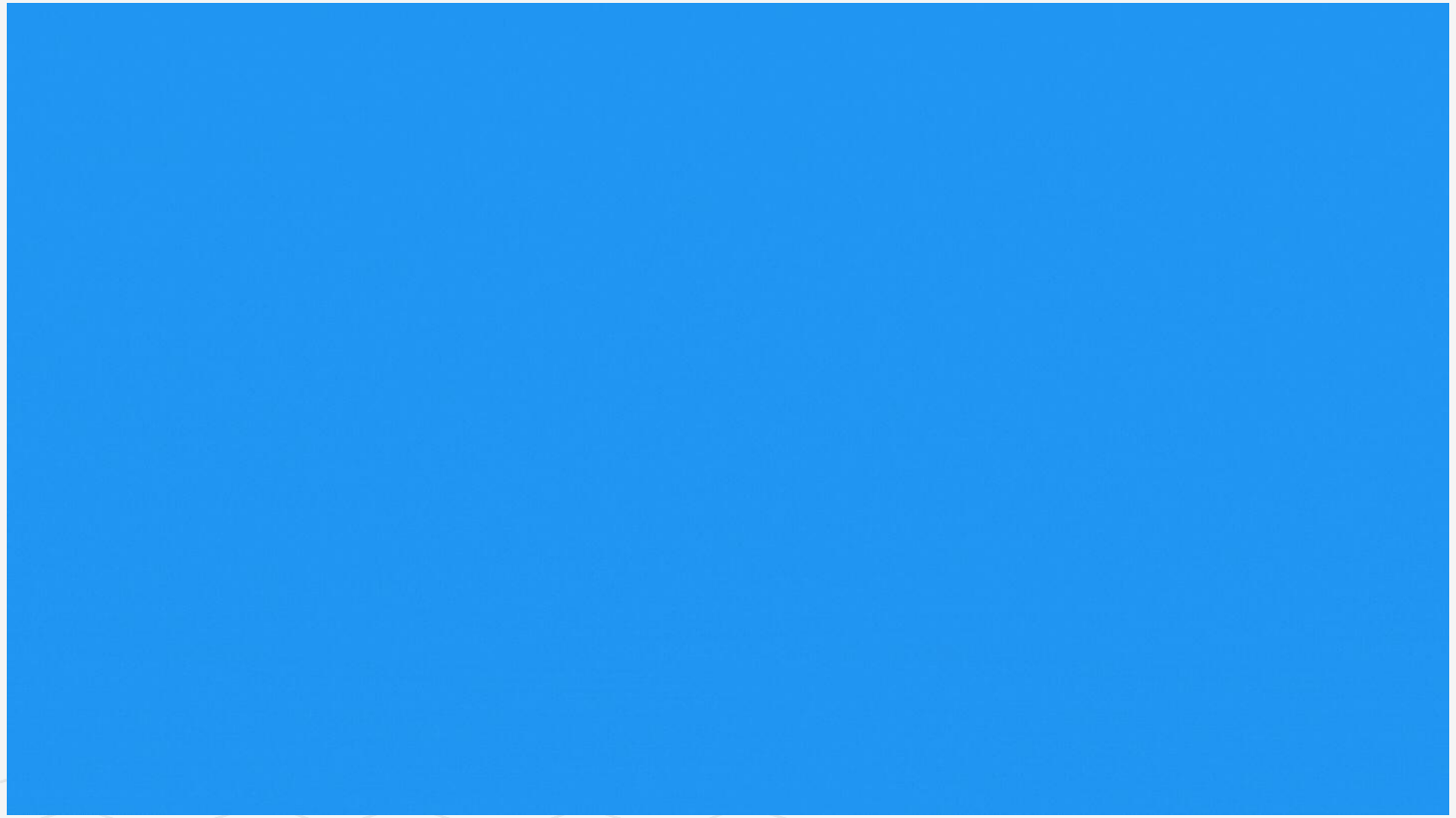
Step-9 Stop

Time and Space Complexity

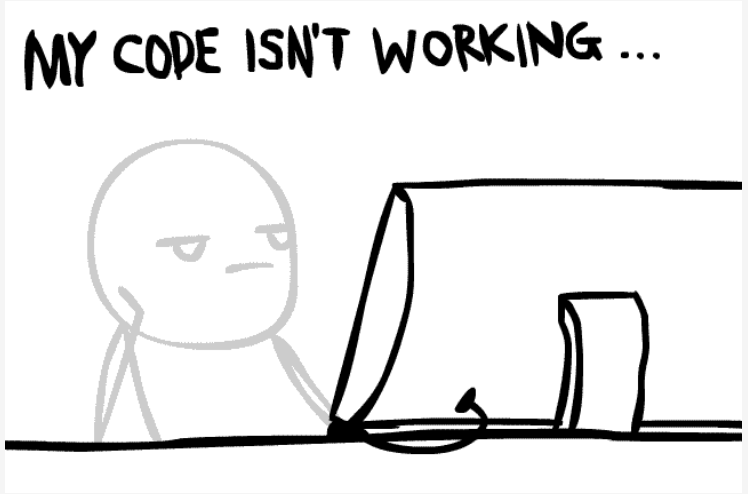
(LABORATORY)

The **time complexity** of an algorithm is basically the running time of a program as a function of the input size.

The **space complexity** of an algorithm is the amount of computer memory that is required during the program execution as a function of the input size.

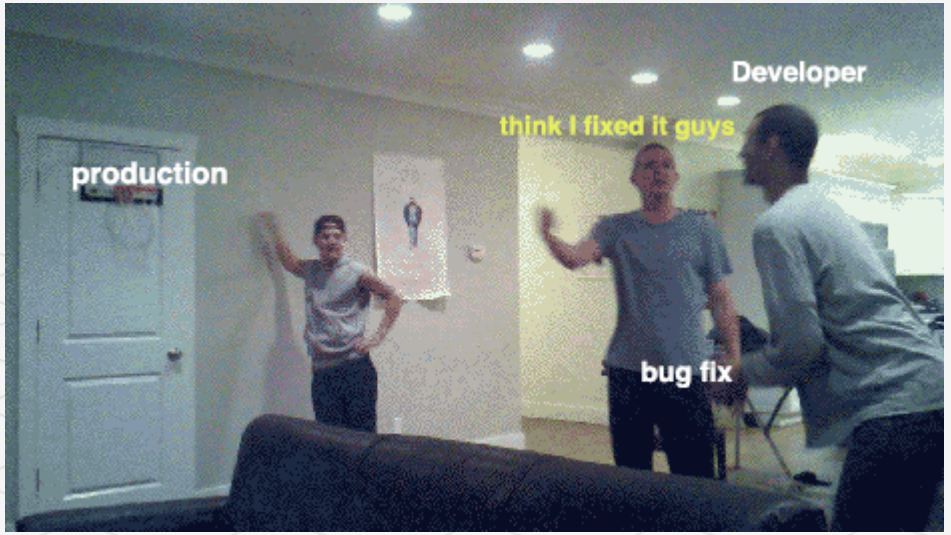


The space needed by a program depends on the following two parts:



Fixed part: It varies from *problem* to *problem*. It includes the space needed for storing instructions, constants, variables, and structured variables (like arrays and structures).

Variable part: It varies from *program* to *program*. It includes the space needed for recursion stack, and for structured variables that are allocated space dynamically during the runtime of a program.



Worst-case, Average-case, Best-case, and Amortized Time Complexity

- **Worst-case running time** - This denotes the behavior of an algorithm with respect to the worst possible case of the input instance.
- The **average-case running time** of an algorithm is an estimate of the running time for an 'average' input.
- **Best-case running time** - The term 'best-case performance' is used to analyze an algorithm under optimal conditions.
- **Amortized running time** refers to the time required to perform a sequence of (related) operations averaged over all the operations performed.



Time–Space Trade-off



The best algorithm to solve a particular problem at hand is no doubt *the one that requires less memory space and takes less time to complete its execution*.

Expressing Time and Space Complexity

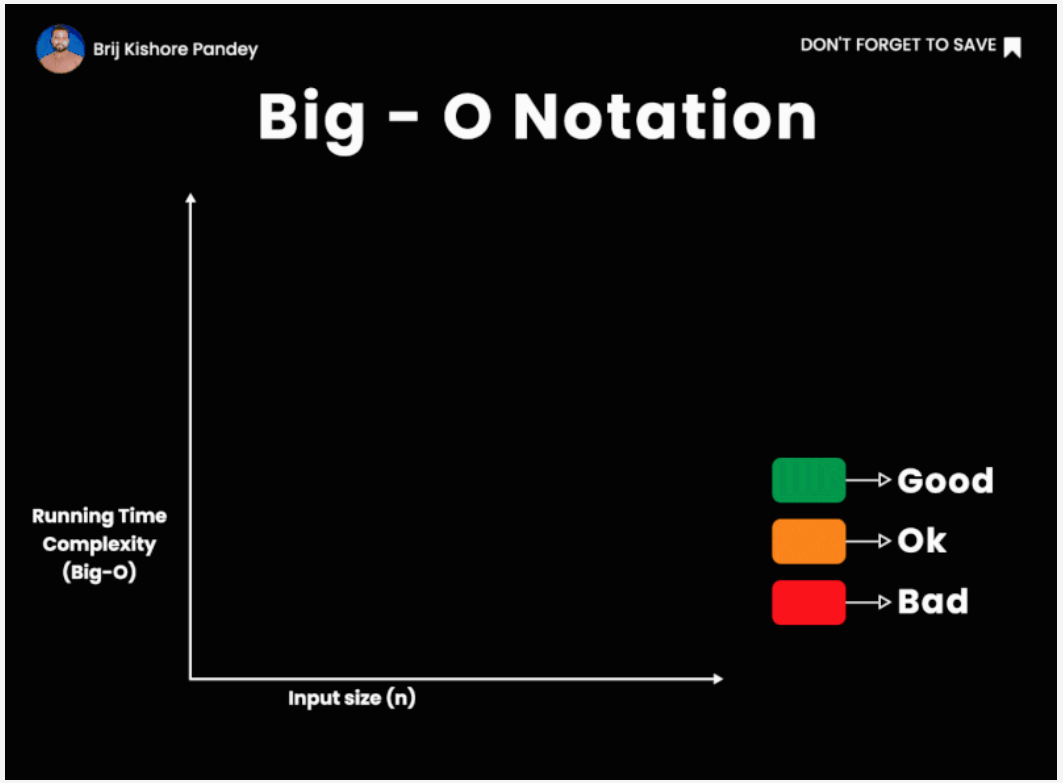
Time and space complexity are written as a **function *f(n)***, where n is the input size. This helps us:

- predict how complexity grows as input increases, and
- compare which algorithm is more efficient.

The most common way to express this is **Big O notation**, which shows the upper bound of complexity.

BIG O NOTATION

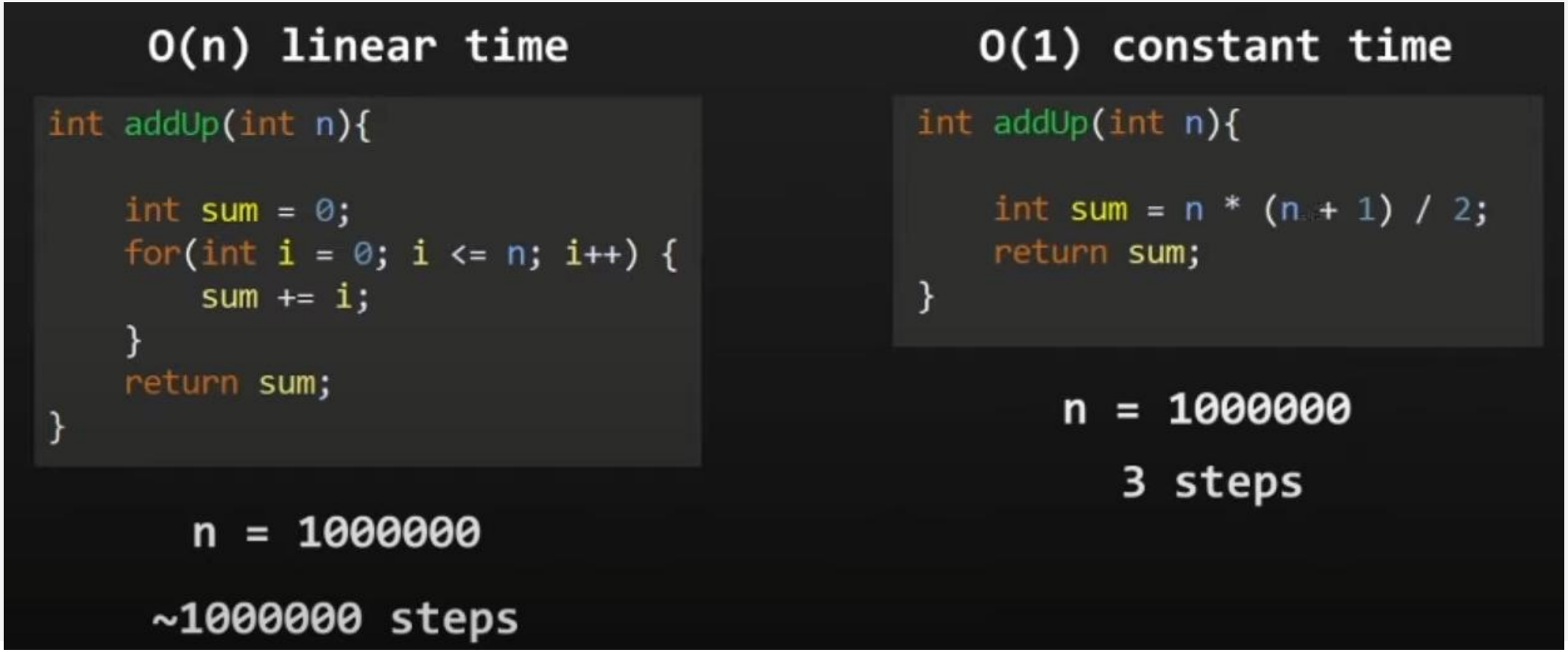
"How code slows as data grows."



1. Describes the performance of an algorithm as the amount of data increases.
2. Machine independent (# steps of completion)
3. Ignore smaller operations
   0(n + 1) -> 0(n)

BIG O NOTATION

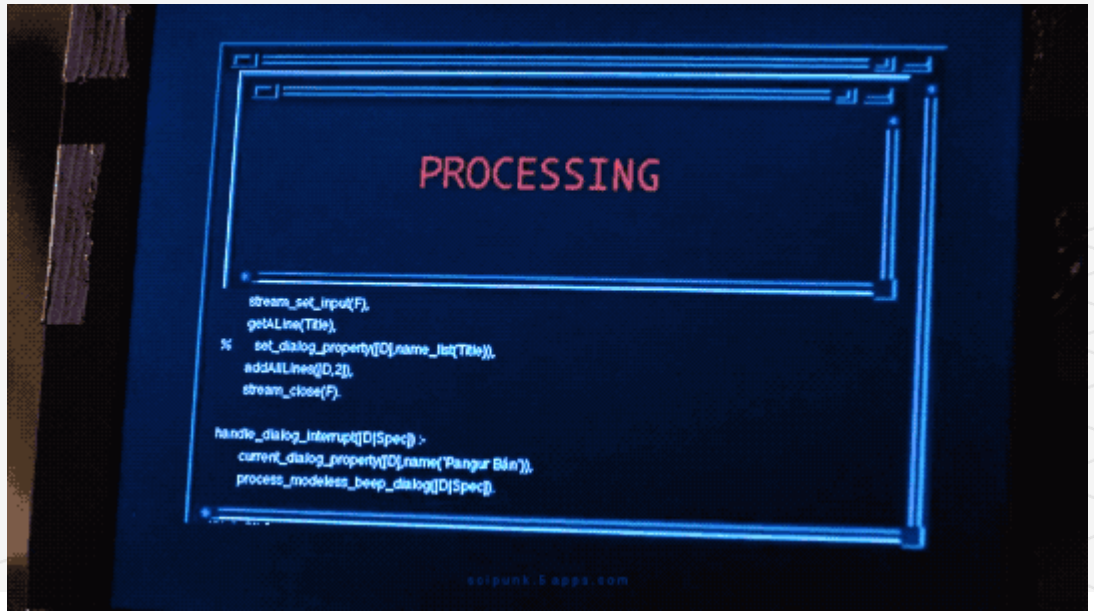
"How code slows as data grows."



Computation Rules

When analyzing an algorithm:

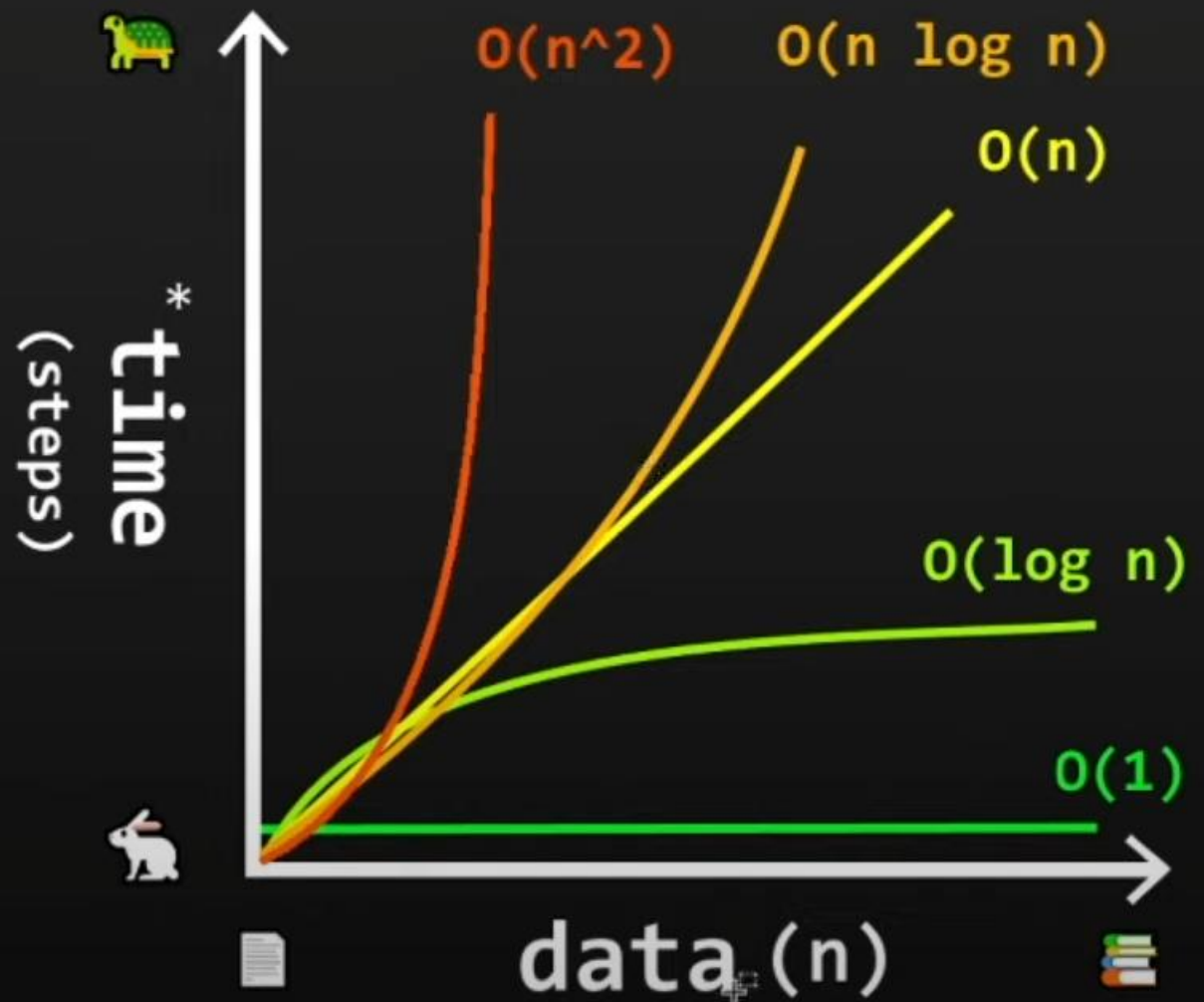
1. **Count the basic operations** (assignments, comparisons, arithmetic).
2. **Focus on the input size (n)**.
3. **Drop constants** (e.g., 2n → 0(n)).
4. **Keep the dominant term only** (e.g., $n^2 + n \rightarrow 0(n^2)$).



Quick Trick to Compute Big O

1. Count loops:
    1. 1 loop → O(n)
    2. 2 nested loops → $O(n^2)$
    3. 3 nested loops → $O(n^3)$
2. Sequential statements → Add them (then drop smaller terms).
3. Drop constants.
4. Keep the **fastest growing term**.

BIG O NOTATION



O(1) = constant time
- random access of an element in an array
- inserting at the beginning of linkedlist

O(log n) = logarithmic time
- binary search

O(n) = linear time
- looping through elements in an array
- searching through a linkedlist

O(n log n) = quasilinear time
- quicksort
- mergesort
- heapsort

O(n^2) = quadratic time
- insertion sort
- selection sort
- bubblesort

Example 1: Constant Time – **O(1)**

```c
#include <stdio.h>
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    printf("%d", arr[0]);   // Just 1 operation
    return 0;
}
```

No matter how big the array is, it always accesses one element.

Example 2: Linear Time – **O(n)**

```c
#include <stdio.h>
int main() {
    int arr[100] = {1,2}, n = 100;
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);   // Runs n times
    }
    return 0;
}
```

Loop runs n times → complexity grows with input size.

Example 3: Quadratic Time – $O(n^2)$

```c
#include <stdio.h>
int main() {
    int arr[100], n = 100;
    for (int i = 0; i < n; i++) {        // n times
        for (int j = 0; j < n; j++) {    // n times
            printf("%d %d\n", i, j);     // n * n times
        }
    }
    return 0;
}
```

Two nested loops → $\mathbf{n \times n = n^2}$

Example 3: Cubic – **O(n3)**

```
void tripleLoop(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                printf("%d %d %d\n", i, j, k);
            }
        }
    }
}
```

Find the maximum number in an array.

```c
#include <stdio.h>

int main() {
    int arr[] = {10, 50, 20, 90, 40};
    int n = 5;
    int max, i, j, isMax;

    max = arr[0];

    for (i = 0; i < n; i++) {
        isMax = 1;
        for (j = 0; j < n; j++) {
            if (arr[j] > arr[i]) {
                isMax = 0;
                break;
            }
        }
        if (isMax == 1) {
            max = arr[i];
        }
    }

    printf("Maximum element is: %d\n", max);
    return 0;
}
```

Maximum element is: 90

Find the maximum number in an array.

```c
#include <stdio.h>

int main() {
    int arr[] = {10, 50, 20, 90, 40};
    int n = 5;
    int max = arr[0];

    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }

    printf("Maximum element is: %d\n", max);
    return 0;
}
```

Maximum element is: 90

Limitations of Big O Notation

1. Some algorithms are too complex to analyze mathematically.
2. Average-case behavior may lack enough data for analysis.
3. Big O shows growth with input size but not actual efficiency or coding effort.
4. It ignores constants (e.g., $O(n^2)$ vs. $O(100000n^2)$), which can matter in real systems.

Other Common Asymptotic notations

Notation	Meaning	Common Use	Example
Big O (O)	Upper bound, worst-case growth	Used in **industry** to describe maximum time/memory an algorithm may need	Linear search is O(n) in worst case
Big Ω (Ω)	Lower bound, best-case growth	Shows how fast an algorithm can be in the best scenario. less common, but useful in **search problems**	Linear search is Ω(1) if item is first
Big Θ (Θ)	Tight bound (average-case or exact growth)	**Academic** use to show precise algorithm complexity	Insertion sort is $\Theta(n^2)$ on average
Little o (o)	Upper bound that is *not tight*	Theoretical comparison to show one grows strictly slower	n is $o(n^2)$
Little ω (ω)	Lower bound that is *not tight*	Theoretical comparison to show one grows strictly faster	n^2 is ω(n)

Thank you